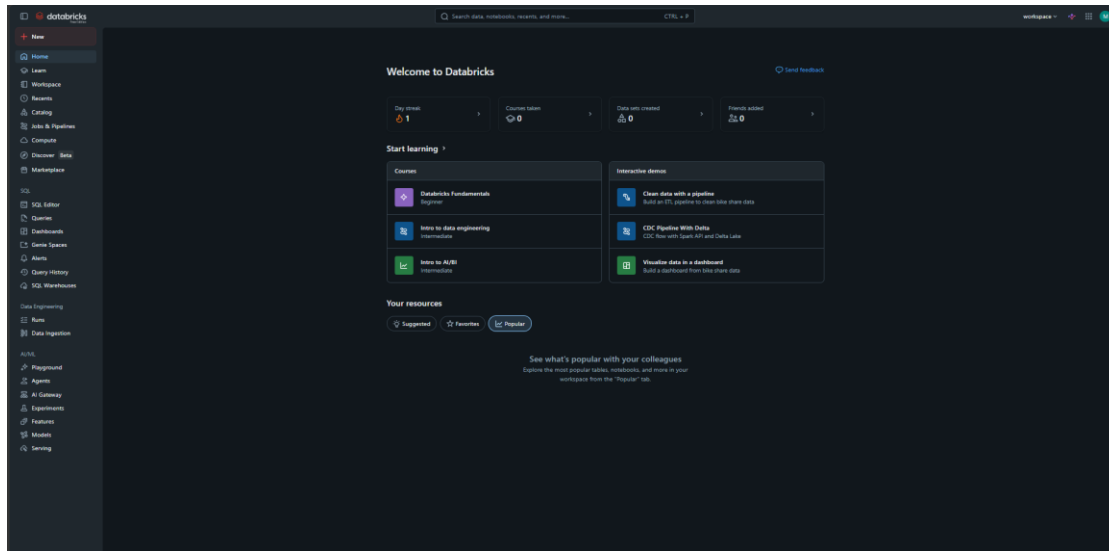


Data Engineering Pipeline using Databricks, AWS S3, Neon PostgreSQL

DATABRICKS SETUP:

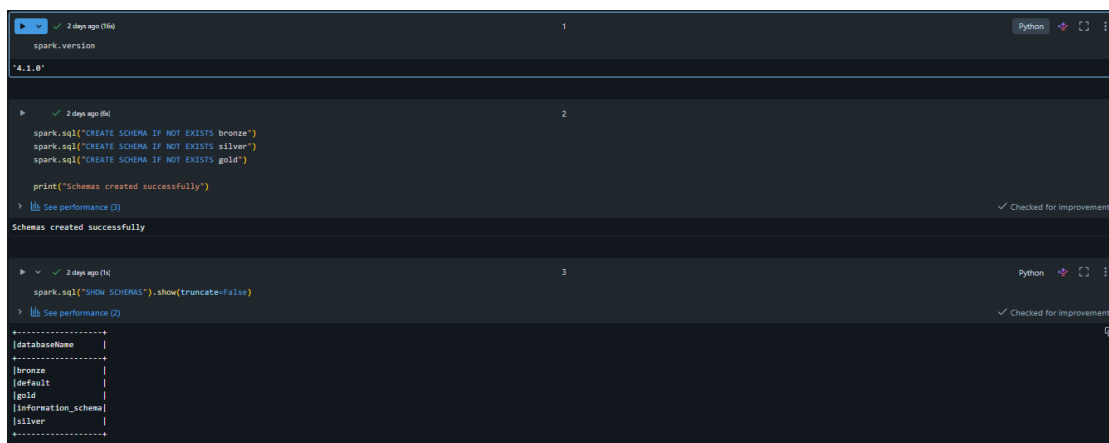
This section shows the initial Databricks workspace setup, catalog configuration, and project environment preparation.



It documents a key step in the implementation process and provides evidence of the successful configuration, execution, or validation performed during this stage.

SCHEMA CREATION:

These notebook screenshots demonstrate the creation of database schemas, catalogs, and foundational tables required for the project.



It documents a key step in the implementation process and provides evidence of the successful configuration, execution, or validation performed during this stage.

CONNECTION OF NEON TO DATABRICKS:

```
4
jdbc_url = "jdbc:postgresql://google.com:5432/test"
print("Ready")

Ready

5
host = "ep-lively-hill-aswbnogx-pooler.c-4.eu-central-1.aws.neon.tech"
database = "neondb"
user = "neondb_owner"
password = "*****"

6
jdbc_url = "jdbc:postgresql://{host}:5432/{database}?sslmode=require"
print(jdbc_url)

jdbc:postgresql://ep-lively-hill-aswbnogx-pooler.c-4.eu-central-1.aws.neon.tech:5432/neondb?sslmode=require

7
spark.read \
  .format("jdbc") \
  .option("url", jdbc_url) \
  .option("database", "drivers") \
  .option("user", user) \
  .option("password", password) \
  .option("driver", "org.postgresql.Driver") \
  .load() \
  .show(5)

> See performance (1) ✓ Checked for improvements

+-----+-----+-----+-----+-----+-----+-----+-----+
|driverid|driverref|number|code|forename|surname|dob|nationality|url|
+-----+-----+-----+-----+-----+-----+-----+-----+
|1|hamilton|44|HAM|Lewis|Hamilton|1985-01-07|British|http://en.wikiped...|
|2|heidfeld|14|HEI|Nick|Heidfeld|1977-05-10|German|http://en.wikiped...|
|3|rosberg|6|ROS|Nico|Rosberg|1985-06-27|German|http://en.wikiped...|
|4|alonso|14|ALO|Fernando|Alonso|1981-07-29|Spanish|http://en.wikiped...|
|5|kovalainen|14|KOV|Heikki|Kovalainen|1981-10-19|Finnish|http://en.wikiped...|
+-----+-----+-----+-----+-----+-----+-----+-----+
only showing top 5 rows
```

These screenshots show how Neon PostgreSQL was connected to Databricks and validated before data ingestion.

The screenshot displays the Databricks workspace interface, specifically the 'Add data' section. The interface is dark-themed and includes a sidebar with navigation options like Home, Learn, Workspace, Recent, Catalog, Jobs & Pipelines, Compute, Discover, and Marketplace. The main area is titled 'Add data' and contains a search bar for data sources. Below this, there are two main sections: 'Files' and 'Data sources'. The 'Files' section has two cards: 'Create or modify table' and 'Upload files to a volume'. The 'Data sources' section is a grid of various data sources, each with a preview button. The 'Ingest from S3' section is highlighted, showing 'Ingestion mode' (One-time or Scheduled) and 'External location' (s3://databricks-warehouse). Below this, there is a table of files to be ingested.

File Name	Size	Last Modified
customer.csv	15.4B KB	15 minutes ago
pt_steps.csv	288.91 KB	15 minutes ago
results.csv	218.41 KB	15 minutes ago

CREATION OF BROZE LAYER:

```
> 2 days ago (7d) 10: bronze races

races_df = spark.read \
    .format("jdbc") \
    .option("url", jdbc_url) \
    .option("dbtable", "races") \
    .option("user", user) \
    .option("password", password) \
    .option("driver", "org.postgresql.Driver") \
    .load()

races_df.write.mode("overwrite").saveAsTable("bronze.races")

> See performance (1) ✓ Checked for improvements
> races_df: pyspark.sql.connect.dataframe.DataFrame = [raceid: integer, year: integer ... 4 more fields]

11: Bronze constructors

spark.sql("DROP TABLE IF EXISTS bronze.constructors")

constructors_df = (
    spark.read
    .option("header", "true")
    .option("inferSchema", "true")
    .csv("s3://f1-dataset-mani/RAM/constructors.csv")
)

constructors_df.write \
    .mode("overwrite") \
    .saveAsTable("bronze.constructors")

print("bronze.constructors refreshed")

> See performance (2) ✓ Checked for improvements
> constructors_df: pyspark.sql.connect.dataframe.DataFrame = [constructorid: integer, constructorRef: string ... 3 more fields]
bronze.constructors refreshed

12: Bronze results

spark.sql("DROP TABLE IF EXISTS bronze.results")

results_df = (
    spark.read
    .option("header", "true")
    .option("inferSchema", "true")
    .csv("s3://f1-dataset-mani/RAM/results.csv")
)

results_df.write \
    .mode("overwrite") \
    .saveAsTable("bronze.results")

print("bronze.results refreshed")

> See performance (2) ✓ Checked for improvements
> results_df: pyspark.sql.connect.dataframe.DataFrame = [resultid: integer, raceid: integer ... 10 more fields]
bronze.results refreshed

13: Bronze pitstops

spark.sql("DROP TABLE IF EXISTS bronze.pit_stops")

pitstops_df = (
    spark.read
    .option("header", "true")
    .option("inferSchema", "true")
    .csv("s3://f1-dataset-mani/RAM/pit_stops.csv")
)

pitstops_df.write \
    .mode("overwrite") \
    .saveAsTable("bronze.pit_stops")

print("bronze.pit_stops refreshed")

> See performance (2) ✓ Checked for improvements
> pitstops_df: pyspark.sql.connect.dataframe.DataFrame = [raceid: integer, driverid: integer ... 5 more fields]
bronze.pit_stops refreshed

8: Bronze drivers

drivers_df = spark.read \
    .format("jdbc") \
    .option("url", jdbc_url) \
    .option("dbtable", "drivers") \
    .option("user", user) \
    .option("password", password) \
    .option("driver", "org.postgresql.Driver") \
    .load()

drivers_df.write.mode("overwrite").saveAsTable("bronze.drivers")

> See performance (1) ✓ Checked for improvements
> drivers_df: pyspark.sql.connect.dataframe.DataFrame = [driverid: integer, driverRef: string ... 7 more fields]

9

spark.sql("""
SELECT *
FROM bronze.drivers
WHERE driverid = 99999
""").show()

> See performance (1) ✓ Checked for improvements
```

driverid	driverRef	number	code	forename	surname	dob	nationality	url
99999	mani	2416	NM	MA	AM	2026-06-03	IND	http://en.wiki.no...

This notebook reads data from the Neon PostgreSQL database and AWS S3 using Spark JDBC connectivity and loads it into the Bronze layer.

CREATION OF SILVER LAYER:

```
15: Silver driver pitstop analysis

spark.sql("""
CREATE OR REPLACE TABLE silver.driver_pitstop_analysis AS
SELECT
    d.driverid,
    CONCAT(d.forename, ' ', d.surname) AS driver_name,
    c.name AS constructor_name,
    ra.year,

    SUM(r.points) AS total_points,
    COALESCE(AVG(CAST(ps.milliseconds AS DOUBLE)/1000),0) AS avg_pitstop_seconds,
    AVG(r.positionorder) AS avg_finish_position

FROM bronze.results r

JOIN bronze.drivers d
    ON r.driverid = d.driverid

JOIN bronze.constructors c
    ON r.constructorid = c.constructorid

JOIN bronze.races ra
    ON r.raceid = ra.raceid

LEFT JOIN bronze.pit_stops ps
    ON r.driverid = ps.driverid
    AND r.raceid = ps.raceid

GROUP BY
    d.driverid,
    CONCAT(d.forename, ' ', d.surname),
    c.name,
    ra.year
""")

> See performance \(1\) ✓ Checked for improvements
DataFrame[num_affected_rows: bigint, num_inserted_rows: bigint]
```

```
14: Silver constructor performance

spark.sql("""
CREATE OR REPLACE TABLE silver.constructor_performance AS
SELECT
    c.constructorid,
    c.name AS constructor_name,
    r.raceid,
    ra.year,
    SUM(r.points) AS total_points,
    SUM(CASE WHEN r.positionorder = 1 THEN 1 ELSE 0 END) AS wins

FROM bronze.results r

JOIN bronze.constructors c
    ON r.constructorid = c.constructorid

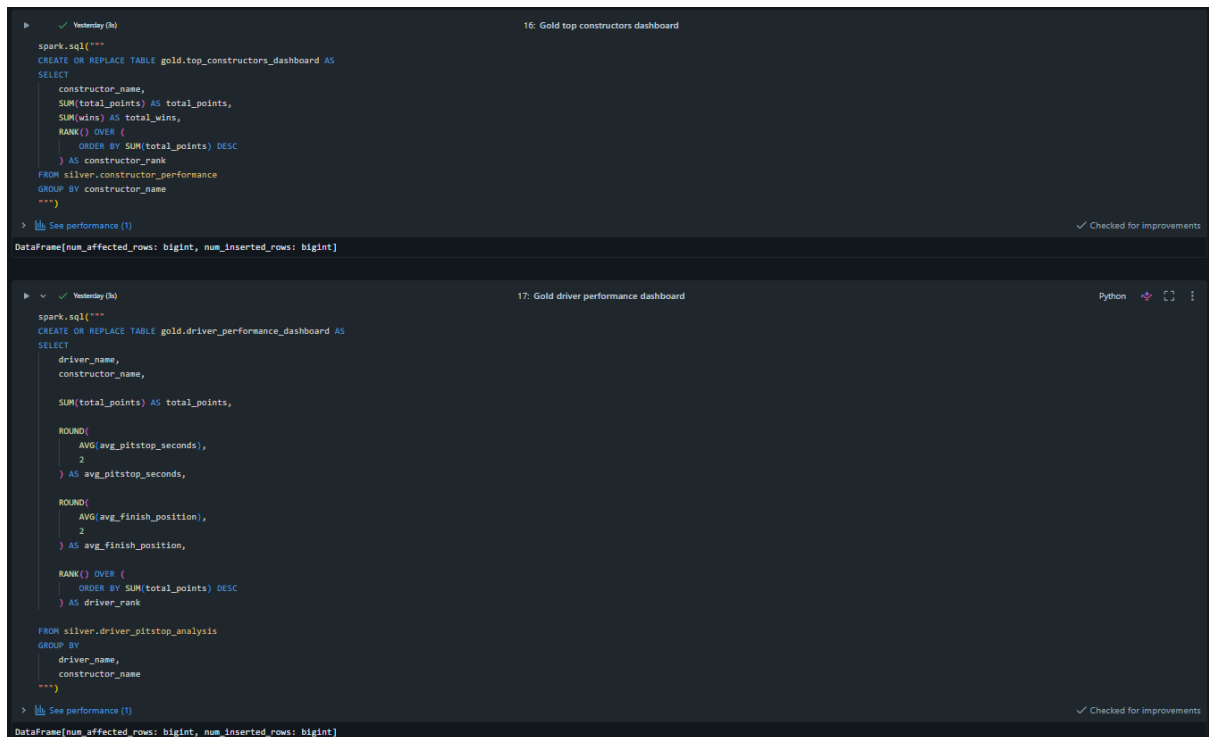
JOIN bronze.races ra
    ON r.raceid = ra.raceid

GROUP BY
    c.constructorid,
    c.name,
    r.raceid,
    ra.year
""")

> See performance \(1\) Optimize
```

This notebook reads data from the Neon PostgreSQL database and AWS S3 using Spark JDBC connectivity and loads it into the Silver layer. The Silver Layer cleans and transforms the Bronze Layer data to create driver pitstop analysis and constructor performance datasets for analytical reporting

CREATION OF GOLD LAYER:



The screenshot displays two Databricks notebooks. The first notebook, titled '16: Gold top constructors dashboard', contains a Spark SQL query that creates a table named 'gold.top_constructors_dashboard'. This query aggregates data from 'silver.constructor_performance' by constructor name, calculating total points and wins, and then ranks them. The second notebook, titled '17: Gold driver performance dashboard', contains a Spark SQL query that creates a table named 'gold.driver_performance_dashboard'. This query aggregates data from 'silver.driver_pitstop_analysis' by driver and constructor names, calculating average pitstop and finish positions, and then ranks them by total points.

```
spark.sql("""
CREATE OR REPLACE TABLE gold.top_constructors_dashboard AS
SELECT
  constructor_name,
  SUM(total_points) AS total_points,
  SUM(wins) AS total_wins,
  RANK() OVER (
    ORDER BY SUM(total_points) DESC
  ) AS constructor_rank
FROM silver.constructor_performance
GROUP BY constructor_name
""")
```

> [See performance \(1\)](#) ✓ Checked for improvements

DataFrame[num_affected_rows: bigint, num_inserted_rows: bigint]

```
spark.sql("""
CREATE OR REPLACE TABLE gold.driver_performance_dashboard AS
SELECT
  driver_name,
  constructor_name,
  SUM(total_points) AS total_points,
  ROUND(
    AVG(avg_pitstop_seconds),
    2
  ) AS avg_pitstop_seconds,
  ROUND(
    AVG(avg_finish_position),
    2
  ) AS avg_finish_position,
  RANK() OVER (
    ORDER BY SUM(total_points) DESC
  ) AS driver_rank
FROM silver.driver_pitstop_analysis
GROUP BY
  driver_name,
  constructor_name
""")
```

> [See performance \(1\)](#) ✓ Checked for improvements

DataFrame[num_affected_rows: bigint, num_inserted_rows: bigint]

The Gold Layer creates dashboard-ready datasets by aggregating and analyzing the transformed data from the Silver Layer.

TOP CONSTRUCTORS PERFORMANCE ☆

Data

Untitled Page

+

Datasets

Top Ten Constructors Rankin...

Top Formula One Construct...

Top Five Constructors by Po...

Rank Table

+ Add data

+ Add SQL dataset

Run

a minute ago

Default (workspace)

Default (default)

SQL

```

1 SELECT
2   constructor_name,
3   total_wins
4 FROM public.top_constructors_dashboard
5 ORDER BY total_wins DESC
6 LIMIT 10

```

+ Add parameter

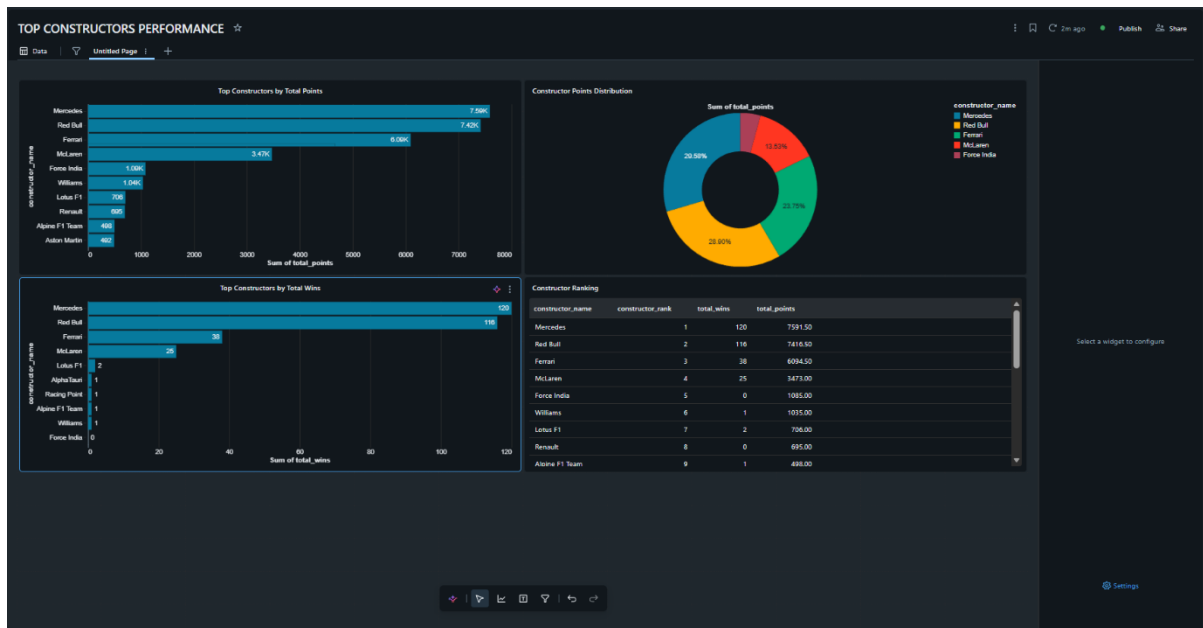
Result Table

Schema

+ Add custom calculation

	constructor_name	total_wins
1	Mercedes	120
2	Red Bull	116
3	Ferrari	38
4	McLaren	25
5	Lotus F1	2
6	Williams	1
7	Alpine F1 Team	1
8	Racing Bulls	1
9	AlphaTauri	1
10	Force India	0

See performance (1)



The Top Constructors Performance Dashboard was created in Databricks by first developing SQL datasets from the curated Gold Layer table (gold.top_constructors_dashboard), which contains aggregated Formula One constructor performance metrics such as constructor rank, total points, and total wins. Separate SQL queries were created to retrieve the top constructors by total points, top constructors by total wins, the top five constructors for points distribution analysis, and a complete constructor ranking table. These datasets were then added to a new Databricks Dashboard as individual visualizations. A horizontal bar chart was used to display the Top Constructors by Total Points, a donut chart was created to show the Constructor Points Distribution among the top teams, another horizontal bar chart was used to visualize the Top Constructors by Total Wins, and a tabular widget was added to present detailed constructor rankings. During dashboard configuration, appropriate fields such as constructor_name, total_points, and total_wins were mapped to the chart axes and measures. After arranging the visualizations into an intuitive layout, the dashboard was published, enabling users to interactively analyze constructor performance. The dashboard ultimately provides a comprehensive view of Formula One constructor rankings, points accumulation, race victories, and overall team performance using data processed through the Bronze, Silver, and Gold layers of the Databricks Medallion Architecture.

OBJECTIVE 2: DRIVER PERFORMANCE AND PIT STOP IMPACT

DRIVER PERFORMANCE AND PIT STOP IMPACT

Data

F1 Dashboard

Datasets

Top 10 Drivers by Total Points

Fastest Average Pit Stops b...

Pit Stop Impact on Finish P...

Driver Ranking Table

+ Add data

+ Add SQL dataset

Run

Just now

Default (workspace)

Default (default)

Edit

```
1 SELECT
2   driver_name,
3   total_points
4 FROM gold_driver_performance_dashboard
5 ORDER BY total_points DESC
6 LIMIT 10;
```

Result Table

Schema

+ Add custom calculation

	driver_name	1.2 total_points
1	Lewis Hamilton	7381.5
2	Max Verstappen	5503.5
3	Sebastian Vettel	3172
4	Nico Rosberg	3004
5	Sebastian Wettel	2596
6	Fernando Alonso	2445
7	Charles Leclerc	2388
8	Valtteri Bottas	2210
9	Daniel Ricciardo	2003
10	Jenson Button	1893

See performance (1)

DRIVER PERFORMANCE AND PIT STOP IMPACT

Data

F1 Dashboard

Datasets

Top 10 Drivers by Total Points

Fastest Average Pit Stops b...

Pit Stop Impact on Finish P...

Driver Ranking Table

+ Add data

+ Add SQL dataset

Run

Just now

Default (workspace)

Default (default)

Edit

```
1 SELECT
2   driver_name,
3   avg_pitstop_seconds,
4 FROM gold_driver_performance_dashboard
5 WHERE avg_pitstop_seconds > 8
6 ORDER BY avg_pitstop_seconds ASC
7 LIMIT 10;
```

Result Table

Schema

+ Add custom calculation

	driver_name	1.2 avg_pitstop_seconds
1	Vitaly Petrov	11.95
2	Timo Glock	11.74
3	Nikita Krasovskiy	11.6
4	Jarno Trulli	11.53
5	Sebastian Buemi	12.01
6	Rubens Barrichello	12.31
7	Jarno Trulli	12.55
8	Pedro de la Rosa	13.58
9	Michael Schumacher	15
10	Lewis Hamilton	15.11

See performance (1)

DRIVER PERFORMANCE AND PIT STOP IMPACT

Data

F1 Dashboard

Top 10 Drivers by Total Points

Fastest Average Pit Stops by Team

Pit Stop Impact on Finish Position

Driver Ranking Table

+ Add data

+ Add SQL dataset

Run

Just now

Default (workspace)

Default (default)

Edit

```
1 SELECT
2   driver_name,
3   avg_pitstop_seconds,
4   avg_finish_position,
5   FROM gold.driver_performance_dashboard
6 GROUP BY avg_pitstop_seconds;
```

+ Add parameter

Result Table

Schema

	driver_name	avg_pitstop_seconds	avg_finish_position
1	Karen Choudhury	0	18
2	Lara & Greal	0	19.52
3	Arvid Lotterer	0	22
4	Nico Hulkenberg	0	14.05
5	Christian Klien	0	20.87
6	Nick Heidfeld	0	12.6
7	Vitantonio Iozzi	0	14.99
8	Sakon Yamamoto	0	13.21
9	Rubens Barrichello	0	8.52
10	Bruno Senna	0	19.28
11	Vitaly Petrov	11.96	12.88
12	Timo Glock	11.74	16.76
13	Heikki Kovalainen	11.9	17.23
14	Jarno Trulli	11.93	13.96
15	Sébastien Buemi	12.01	13.82
16	Rubens Barrichello	12.31	12.59
17	Jarno Trulli	12.55	18.58
18	Pedro de la Rosa	13.58	14.25
19	Michael Schumacher	15	11.23
20	Lewis Hamilton	15.11	7.1
21	Kimi Räikkönen	16.14	15.2
22	Felipe Massa	17.1	8.49

See performance (1)

+ Add custom calculation

DRIVER PERFORMANCE AND PIT STOP IMPACT

Data

F1 Dashboard

Top 10 Drivers by Total Points

Fastest Average Pit Stops by Team

Pit Stop Impact on Finish Position

Driver Ranking Table

+ Add data

+ Add SQL dataset

Run

Just now

Default (workspace)

Default (default)

Edit

```
1 SELECT
2   driver_name,
3   driver_name,
4   constructor_name,
5   total_points,
6   ROUND(avg_pitstop_seconds, 1) AS avg_pitstop_seconds,
7   ROUND(avg_finish_position, 1) AS avg_finish_position,
8   FROM gold.driver_performance_dashboard
9 GROUP BY driver_name;
```

+ Add parameter

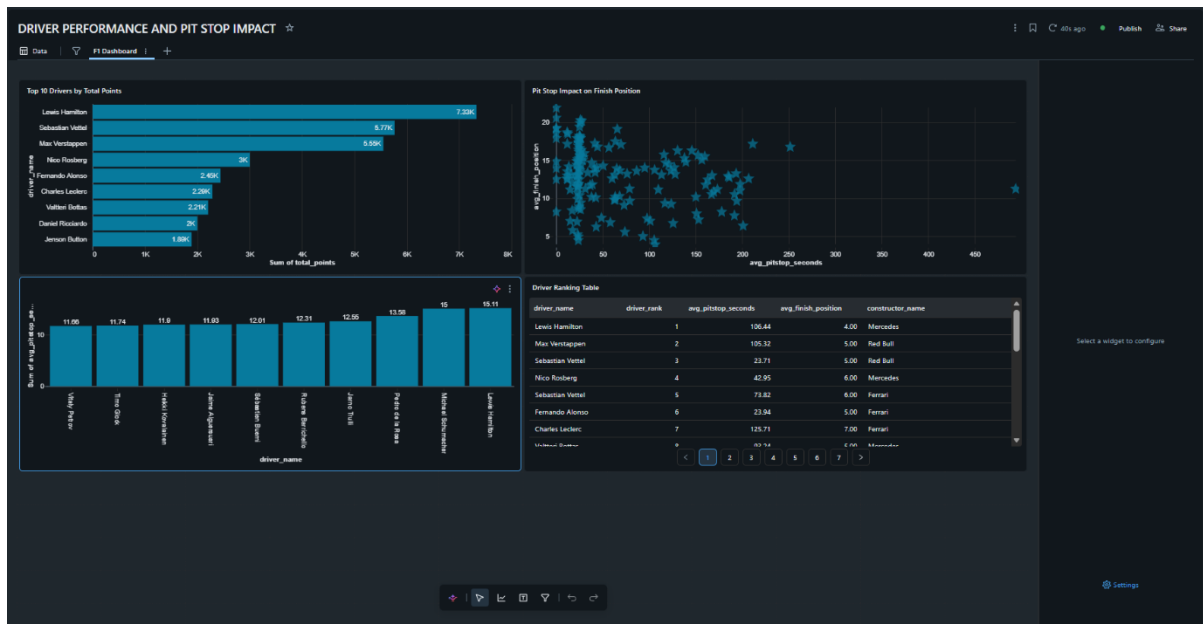
Result Table

Schema

	driver_rank	driver_name	constructor_name	total_points	avg_pitstop_seconds	avg_finish_position
1	1	Lewis Hamilton	Mercedes	7331.5	106.44	4
2	2	Max Verstappen	Red Bull	5553.5	105.32	5
3	3	Sébastien Vettel	Red Bull	3172	23.71	5
4	4	Nico Rosberg	Mercedes	3004	42.95	6
5	5	Sébastien Vettel	Ferrari	2596	73.82	6
6	6	Fernando Alonso	Ferrari	2445	23.94	5
7	7	Charles Leclerc	Ferrari	2288	125.71	7
8	8	Valtteri Bottas	Mercedes	2210	93.24	5
9	9	Daniel Ricciardo	Red Bull	2003	58.17	7
10	10	Jenson Button	McLaren	1893	38	10
11	11	Mark Webber	Red Bull	1989	17.66	6
12	12	Sergio Pérez	Red Bull	1784	113.15	7
13	13	Lando Norris	McLaren	1712	150.84	8
14	14	Carlos Sainz	Ferrari	1098.5	200.94	6
15	15	Kimi Räikkönen	Ferrari	1482	48.6	8
16	16	Lewis Hamilton	McLaren	1281	15.11	7
17	17	George Russell	Mercedes	1226	97.69	7
18	18	Felipe Massa	Ferrari	1008	17.1	8
19	19	Kimi Räikkönen	Lotus F1	827	23.04	5
20	20	Valtteri Bottas	Williams	785	63.41	10
21	21	Sergio Pérez	Force India	720	62.4	10
22	22	Felipe Massa	Williams	676	68.14	10

See performance (1)

+ Add custom calculation



The Driver Performance and Pit Stop Impact Dashboard was created in Databricks by first preparing multiple SQL datasets from the curated Gold Layer table (gold.driver_performance_dashboard), which contains aggregated driver performance metrics derived from the Bronze and Silver layers. Separate SQL queries were developed to support each visualization within the dashboard. The first dataset retrieves the top 10 drivers based on total championship points and is used to create a horizontal bar chart highlighting the highest-performing drivers. The second dataset identifies the drivers with the fastest average pit stop times by selecting and sorting drivers based on their average pit stop duration, which is then visualized using a bar chart. The third dataset retrieves average pit stop times and average finishing positions for all drivers, forming the basis of a scatter plot that helps analyze the relationship between pit stop performance and race results. Finally, a ranking dataset was created containing driver rank, driver name, constructor name, total points, average pit stop time, and average finishing position, which is displayed in a tabular format to provide detailed driver statistics.

After creating and validating each SQL query, the datasets were added to a Databricks Dashboard and connected to appropriate visualizations. The fields from each dataset were mapped to chart axes, measures, and labels to ensure meaningful representation of the data. The visualizations were then arranged into a dashboard layout consisting of the Top 10 Drivers by Total Points chart, the Fastest Average Pit Stops chart, the Pit Stop Impact on Finish Position scatter plot, and the Driver Ranking Table. Once the dashboard design was finalized, it was published within Databricks, enabling interactive exploration of driver performance metrics. The dashboard provides valuable insights into driver rankings, championship points accumulation, pit stop efficiency, and the impact of pit stop performance on race finishing positions, supporting data-driven analysis of Formula One racing performance.

JOB AUTOMATIN IN DATABRICKS:

The screenshot shows the Databricks Jobs & Pipelines interface. A new job named 'F1 PIPELINE' is being configured. The job is currently in a 'New' state. The configuration panel on the right shows the following details:

- Job ID:** 801672183707501
- Creator:** main@main@002122@gmail.com
- Run as:** main@main@002122@gmail.com
- Description:** Add description
- Language:** Python
- Performance optimization:** On
- Schedules & Triggers:** Paused - Every 2 hours. Buttons: Edit trigger, Resume, Delete.
- Job parameters:** No job parameters are defined for this job. Button: Edit parameters.
- Compute:** Serverless. Buttons: Swap, Query history.
- Tags:** Add tag.
- Job notifications:** No notifications. Button: Edit notifications.
- Duration and streaming backfill thresholds:**

The main workspace area shows a graph with two tasks: 'F1_pipeline' (a cluster task) and 'TOP_CONSTRUCTORS' (a table task). The 'F1_pipeline' task is currently selected.

The screenshot shows the Databricks Jobs & Pipelines interface with a list of runs for the 'F1 PIPELINE' job. The 'Runs' tab is selected, showing a table of runs with columns: Run ID, Start time, and Status. The table contains 10 runs, all of which are 'Succeeded'.

Run ID	Start time	Status
990970202095405	Jun 21, 2026, 01:13 AM	Succeeded
227442354791485	Jun 21, 2026, 01:09 AM	Succeeded
6714709963088	Jun 21, 2026, 12:57 PM	Succeeded
610872479191008	Jun 21, 2026, 01:14 PM	Succeeded
1095902761912247	Jun 21, 2026, 01:14 PM	Succeeded
243881706783712	Jun 21, 2026, 01:14 PM	Succeeded
325842118544658	Jun 21, 2026, 01:14 PM	Succeeded
854568102739868	Jun 21, 2026, 01:14 PM	Succeeded
5411285187731	Jun 21, 2026, 01:14 PM	Succeeded
51971468488727	Jun 21, 2026, 11:14 AM	Succeeded
960808143489115	Jun 21, 2026, 09:14 AM	Succeeded
60385156551208	Jun 21, 2026, 07:01 AM	Succeeded
91893953200301	Jun 21, 2026, 07:01 AM	Succeeded
66270419738333	Jun 21, 2026, 02:01 AM	Succeeded
61138121987828	Jun 21, 2026, 01:01 AM	Succeeded
180101807780245	Jun 21, 2026, 01:01 AM	Succeeded

A 'Schedules & Triggers' dialog box is open, showing the following configuration:

- Trigger Status:** Active
- Trigger type:** Scheduled
- Schedule type:** Interval
- Periodic:** Every 2 Hours

The screenshot shows the Databricks Jobs & Pipelines interface with a 'New Job' configuration screen. The job is named 'New Job 2026-06-23 01:18:15'. The configuration panel on the right shows the following details:

- Job ID:** 87138776598105
- Creator:** main@main@002122@gmail.com
- Run as:** main@main@002122@gmail.com
- Description:** Add description
- Language:** Python
- Performance optimization:** Off
- Schedules & Triggers:** None. Button: Add trigger.
- Job parameters:** No job parameters are defined for this job. Button: Edit parameters.
- Tags:** Add tag.
- Job notifications:** No notifications. Button: Edit notifications.
- Duration and streaming backfill thresholds:** No thresholds defined. Button: Add metric threshold.
- Permissions:** main@main@002122@gmail.com is Owner.

The main workspace area shows a 'Add your first task' dialog box with the following options:

- Dashboard:** Refresh a previously configured dashboard and send it to a subscriber.
- Readback:** Run a readback.
- + Add another task link**

These screenshots show the Databricks Job Scheduling and Automation process used to automate the execution of the F1 data pipeline. After the Bronze, Silver, and Gold layer notebooks were successfully developed and tested, a Databricks Job named F1 PIPELINE was created through the Jobs & Pipelines interface. Initially, a new job was configured by selecting the notebook task option and linking it to the pipeline notebook responsible for executing the complete ETL workflow. The job was configured to run on a Serverless Compute environment, eliminating the need for manual cluster management and ensuring efficient execution of the data pipeline.

Once the job was created, a schedule was configured using the Schedules & Triggers settings. The trigger type was set to Scheduled, and an interval-based schedule was defined to automatically execute the pipeline every 2 hours. This configuration enables Databricks to continuously monitor and process newly available data from both Amazon S3 and Neon PostgreSQL without requiring manual intervention. After the schedule was activated, the job execution history became available in the Runs section, where each execution can be monitored along with its status, duration, start time, and completion details.

The final job workflow contains the main F1_pipeline task, which automatically executes the ETL process and refreshes the Gold Layer datasets. Once the pipeline completes successfully, the downstream dashboards, including Driver Performance and Pit Stop Impact and Top Constructors Performance, are refreshed with the latest processed data. This automated scheduling mechanism ensures that any new records added to the source systems are regularly ingested, transformed, and made available for analytical reporting, thereby enabling near real-time business intelligence and performance monitoring within Databricks.

TESTING WITH NEW DATAS:

```
results.csv
File Edit View
26738,1143,859,215,17,14,14,0.0,57,1:26.076,226.641,1
26739,1143,848,3,16,15,15,0.0,56,1:25.559,228.011,11
26740,1143,807,210,18,\N,16,0.0,39,1:25.767,227.458,20
26741,1143,815,9,9,\N,17,0.0,38,1:25.288,228.735,20
26742,1143,840,117,15,\N,18,0.0,8,1:30.935,214.531,130
26743,1143,861,3,19,\N,19,0.0,0,\N,\N,4
26744,1143,839,214,20,\N,20,0.0,0,\N,\N,4
26745,1144,846,1,1,1,1,25.0,58,1:27.438,217.429,1
26746,1144,832,6,3,2,2,18.0,58,1:27.765,216.619,1
26747,1144,844,6,19,3,3,15.0,58,1:28.018,215.996,1
26748,1144,1,131,16,4,4,12.0,58,1:27.278,217.828,1
26749,1144,847,131,6,5,5,10.0,58,1:28.195,215.563,1
26750,1144,830,9,4,6,6,8.0,58,1:27.765,216.619,1
26751,1144,842,214,5,7,7,6.0,58,1:29.251,213.012,1
26752,1144,807,210,7,8,8,4.0,58,1:29.152,213.249,1
26753,1144,4,117,8,9,9,2.0,58,1:27.948,216.168,1
26754,1144,857,1,2,10,10,1.0,58,1:27.690,216.804,1
26755,1144,848,3,18,11,11,0.0,57,1:29.438,212.567,11
26756,1144,852,215,11,12,12,0.0,57,1:29.200,213.134,11
26757,1144,855,15,15,13,13,0.0,57,1:27.982,216.085,11
26758,1144,840,117,13,14,14,0.0,57,1:28.604,214.568,11
26759,1144,862,214,17,15,15,0.0,57,1:29.121,213.323,11
26760,1144,825,210,14,16,16,0.0,57,1:25.637,222.002,11
26761,1144,859,215,12,17,17,0.0,55,1:28.751,214.212,5
26762,1144,822,15,9,\N,18,0.0,30,1:29.482,212.462,130
26763,1144,861,3,20,\N,19,0.0,26,1:29.411,212.631,5
26764,1144,815,9,10,\N,20,0.0,0,\N,\N,4
999999,1144,99999,99999,1,1,1,5000,1,1:20.000,250.000,1
Ln 6438, Col 56 | 55 of 3,26,096 charact | Plain text | 100% | Unix (LF) | UTF-8
```

```
pit_stops.csv
File Edit View
raceId,driverId,stop,lap,time,duration,milliseconds
1144,99999,1,1,00:00:01,0.5,500
841,153,1,1,17:05:23,26.898,26898
841,30,1,1,17:05:52,25.021,25021
841,17,1,11,17:20:48,23.426,23426
841,4,1,12,17:22:34,23.251,23251
841,13,1,13,17:24:10,23.842,23842
841,22,1,13,17:24:29,23.643,23643
841,20,1,14,17:25:17,22.603,22603
841,814,1,14,17:26:03,24.863,24863
841,816,1,14,17:26:50,25.259,25259
841,67,1,15,17:27:34,25.342,25342
841,2,1,15,17:27:41,22.994,22994
841,1,1,16,17:28:24,23.227,23227
841,808,1,16,17:28:39,24.535,24535
841,3,1,16,17:29:00,23.716,23716
841,155,1,16,17:29:06,24.064,24064
841,16,1,16,17:29:08,25.978,25978
841,15,1,16,17:29:49,24.899,24899
841,18,1,17,17:30:24,16.867,16867
841,153,2,17,17:31:06,24.463,24463
841,5,1,17,17:31:11,24.865,24865
841,30,2,17,17:32:08,23.988,23988
841,10,1,18,17:33:02,23.792,23792
841,18,2,19,17:33:53,23.303,23303
841,815,1,23,17:40:27,23.438,23438
841,22,2,23,17:40:45,37.856,37856
841,17,2,26,17:44:29,22.520,22520
Ln 2, Col 32 | 31 of 3,98,267 characte | Plain text | 100% | Unix (LF) | UTF-8
```

constructors.csv

FileEditView

191,brabham-repco,Brabham-Repco,British,http://en.wikipedia.org/wiki/Brabham
192,cooper-ferrari,Ferrari,British,http://en.wikipedia.org/wiki/Cooper_Car_Company
193,cooper-ats,Cooper-ATS,British,http://en.wikipedia.org/wiki/Cooper_Car_Company
194,mclaren-brm,McLaren-BRM,British,http://en.wikipedia.org/wiki/McLaren_(racing)
195,cooper-brm,Cooper-BRM,British,http://en.wikipedia.org/wiki/Cooper_Car_Company
196,matra-ford,Matra-Ford,French,http://en.wikipedia.org/wiki/Matra
197,brm-ford,BRM-Ford,British,http://en.wikipedia.org/wiki/BRM
198,mclaren-alfa_romeo,McLaren-Alfa Romeo,British,http://en.wikipedia.org/wiki/McLaren_(racing)
199,march-alfa_romeo,March-Alfa Romeo,British,http://en.wikipedia.org/wiki/March_Engineering
200,march-ford,March-Ford,British,http://en.wikipedia.org/wiki/March_Engineering
201,lotus-pw,Lotus-Pratt & Whitney,British,http://en.wikipedia.org/wiki/Team_Lotus
202,shadow-ford,Shadow-Ford,British,http://en.wikipedia.org/wiki/Shadow_Racing_Cars
203,shadow-matra,Shadow-Matra,British,http://en.wikipedia.org/wiki/Shadow_Racing_Cars
204,brabham-alfa_romeo,Brabham-Alfa Romeo,British,http://en.wikipedia.org/wiki/Brabham
205,lotus_racing,Lotus,Malaysian,http://en.wikipedia.org/wiki/Lotus_Racing
206,marussia,Marussia,Russian,http://en.wikipedia.org/wiki/Marussia_F1
207,caterham,Caterham,Malaysian,http://en.wikipedia.org/wiki/Caterham_F1
208,lotus_f1,Lotus F1,British,http://en.wikipedia.org/wiki/Lotus_F1
209,manor,Manor Marussia,British,http://en.wikipedia.org/wiki/Manor_Motorsport
210,haas,Haas F1 Team,American,http://en.wikipedia.org/wiki/Haas_F1_Team
211,racing_point,Racing Point,British,http://en.wikipedia.org/wiki/Racing_Point_F1_Team
213,alphaTauri,AlphaTauri,Italian,http://en.wikipedia.org/wiki/Scuderia_AlphaTauri
214,alpine,Alpine F1 Team,French,http://en.wikipedia.org/wiki/Alpine_F1_Team
215,rb,RB F1 Team,Italian,http://en.wikipedia.org/wiki/RB_Formula_One_Team
99999,test_constructor,Test Constructor,German,http://testconstructor.com

Ln 214, Col 7473 of 15,856 character:Plain text100%Unix (LF)UTF-8

minimiseran2002724@gmail.comFree

AI OKAsk AIUpgrade

PROJECT

Dashboard

Branches

Integrations

Settings

SEARCH

production

Overview

Monitoring

SQL Editor

Tables

Backup & Restore

Data Masking

APP BACKEND

Data API

Auth

Refer a friend

Feedback

E Collapse menu

Tables

production

neondb

Schema

public

Search

drivers

Tapes

DATA

STRUCTURE

Filters

Sort

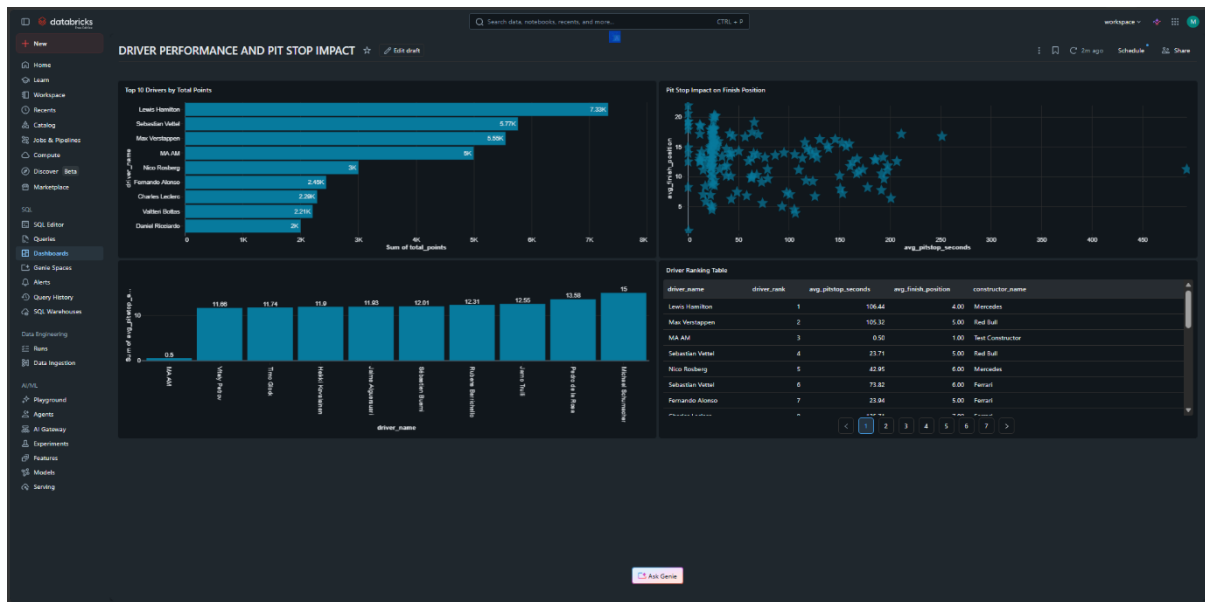
Columns

+ Add record

Delete 2 records

7 rows851 - 853 of 853

id	driver_name	driver_number	team	nationality	url
852	Tsunoda	22	TSU	Yuki	Tsunoda
853	Mazda	9	MZ	Walter	Mazda
854	Wick-Schumacher	47	MS	Wick	Schumacher
855	Zhu	24	ZHU	Georgy	Zhu
856	de Vries	21	DFV	Nyck	de Vries
857	Plastri	81	PIA	Oscar	Plastri
858	Sargeant	2	SAR	Logan	Sargeant
859	Liam	38	LAM	Liam	Liam
860	Oliver	38	OLA	Oliver	Oliver
861	Colapinto	43	COL	Francis	Colapinto
862	Douglas	81	DOD	Jack	Douglas
9416	test_driver	99	TSU	Test	Driver
99999	max1	1416	TM	AM	AM



These screenshots demonstrate the end-to-end data validation and dashboard refresh process implemented in the Formula One analytics project. To verify that the automated ETL pipeline correctly processes newly arriving data, new records were manually inserted into both source systems. In the Neon PostgreSQL database, a new driver record was added with a unique driverId (99999), along with associated driver details such as name, nationality, date of birth, and reference information. Simultaneously, new records were appended to the source CSV files stored in Amazon S3, including results.csv, pit_stops.csv, and constructors.csv. These records were intentionally assigned high values for points and performance metrics to make their impact easily visible in downstream analytics and dashboard visualizations.

After the new source records were added, the Databricks job scheduler automatically executed the Bronze, Silver, and Gold layer transformations according to the configured schedule. The Bronze layer ingested the updated data from Neon and Amazon S3, the Silver layer performed cleansing and standardization of the records, and the Gold layer generated business-ready analytical datasets. Once processing was completed, the dashboards were automatically refreshed using the updated Gold Layer tables.

The validation was confirmed through the Driver Performance and Pit Stop Impact Dashboards, where the newly added driver "MA AM" appeared among the top-ranked drivers due to the large points value inserted into the source data. The driver also appeared in the ranking table with the associated constructor information and performance metrics, demonstrating that data added to both Neon PostgreSQL and Amazon S3 successfully propagated through the entire ETL pipeline. This verification process confirms that the implemented architecture supports automated data ingestion, transformation, and dashboard refreshes, ensuring that newly arriving records are accurately reflected in analytical reports without manual intervention.

SUMMARY :

This project implements an end-to-end Formula One Data Engineering and Analytics Pipeline using the Medallion Architecture (Bronze, Silver, and Gold layers) on Databricks. The objective of the project is to ingest data from multiple sources, transform it into business-ready datasets, and generate interactive dashboards for performance analysis.

The pipeline integrates data from Amazon S3 and Neon PostgreSQL, where raw Formula One datasets such as drivers, constructors, race results, races, and pit stop information are stored. Databricks is used as the central processing platform to ingest the data into the Bronze Layer, preserving the raw source records. The data is then cleaned, standardized, and validated in the Silver Layer to improve data quality and consistency. Finally, the Gold Layer combines and aggregates the transformed datasets to create analytical tables optimized for reporting and business intelligence.

Using the Gold Layer datasets, interactive dashboards were developed in Databricks to analyze driver performance, pit stop efficiency, and constructor performance. The dashboards provide insights such as top drivers by championship points, fastest average pit stops, pit stop impact on finishing position, constructor rankings, total wins, and points distribution. These visualizations enable users to explore Formula One performance metrics through an intuitive and interactive interface.

To ensure continuous data availability, the entire ETL workflow was automated using Databricks Jobs and Scheduling, with the pipeline configured to execute every two hours. New records added to Amazon S3 and Neon PostgreSQL are automatically detected, processed through the Bronze, Silver, and Gold layers, and reflected in the dashboards without manual intervention. Validation testing was performed by inserting new driver, constructor, race result, and pit stop records, confirming that the pipeline successfully updates analytical outputs and dashboard visualizations.

Overall, the project demonstrates the implementation of a modern cloud-based data engineering solution that combines data ingestion, transformation, automation, and visualization to deliver real-time analytical insights for Formula One racing data.